



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 5:

BIT PERMUTATION-PART V



INTRODUCCIÓN A LA CRIPTOGRAFÍA

Profesora || Sandra Díaz Santiago

Grupo || 6CV4

Alumno || Areli Alejandra Guevara Badillo

Noviembre 14, 2025

1. MULTIPLICACIÓN EN $GF(2^n)$

EJERCICIO 1

Diseña e implementa una función que reciba como parámetro un elemento $f(x) \in GF(2^8)$. Utiliza una variable unsigned char para almacenar los coeficientes de $f(x)$. Tu función debe calcular $x*f(x)$ mód $(x^8+x^5+x^3+x+1)$ e imprimir el valor de $f(x)$ y el resultado en hexadecimal.

Código:

```
// Función que multiplica un polinomio por x en GF(2^8)
unsigned char multX(unsigned char fx) {
    unsigned char r = 0x2B;
    unsigned char p = 0;

    int bms = fx >> 7;

    if (bms == 0) {
        p = fx << 1;
    } else {
        p = (fx << 1) ^ r;
    }
    return p;
}
```

Ejecución:

```
Dame f(x) (en hexadecimal):23
f(x) = 0x23
x * f(x) = 0x46

Dame f(x) (en hexadecimal):01
f(x) = 0x01
x * f(x) = 0x02

Dame f(x) (en hexadecimal):02
f(x) = 0x02
x * f(x) = 0x04

Dame f(x) (en hexadecimal):03
f(x) = 0x03
x * f(x) = 0x06

Dame f(x) (en hexadecimal):56
f(x) = 0x56
x * f(x) = 0xac
```

EJERCICIO 2

Diseña e implementa una función que reciba como parámetro dos elementos $f(x), g(x) \in GF(2^8)$. Utiliza variables tipo unsigned char para almacenar los coeficientes de $f(x)$ y $g(x)$.

Tu función debe calcular: $f(x) * g(x) \bmod (x^8 + x^5 + x^3 + x + 1)$ e imprimir los valores de $f(x)$, $g(x)$ y el resultado de la multiplicación en hexadecimal.

Código:

```
// Función que multiplica dos polinomios en GF(2^8)
unsigned char multCampoB(unsigned char fx, unsigned char gx) {
    unsigned char resp = 0x00;
    unsigned char arrg[8];
    unsigned char arrgGx[8];

    for (int i = 0; i < 8; i++) {
        arrgGx[i] = (gx >> i) & 1;
    }

    arrg[0] = fx;
    for (int i = 1; i < 8; i++) {
        arrg[i] = multX(arrg[i - 1]);
    }

    for (int i = 0; i < 8; i++) {
        if (arrgGx[i] == 1) {
            resp ^= arrg[i];
        }
    }

    return resp;
}
```

Ejecución:

```
Dame f(x) (en hexadecimal):57
Dame g(x) (en hexadecimal):83
f(x) = 0x57
g(x) = 0x83
f(x) * g(x) = 0x90
```

```
Dame f(x) (en hexadecimal):A5
Dame g(x) (en hexadecimal):3F
f(x) = 0xa5
g(x) = 0x3f
f(x) * g(x) = 0x36
```

```
Dame f(x) (en hexadecimal):FF
Dame g(x) (en hexadecimal):13
f(x) = 0xff
g(x) = 0x13
f(x) * g(x) = 0x78
```

```
Dame f(x) (en hexadecimal):12
Dame g(x) (en hexadecimal):34
f(x) = 0x12
g(x) = 0x34
f(x) * g(x) = 0x55
```

```
Dame f(x) (en hexadecimal):7F
Dame g(x) (en hexadecimal):01
f(x) = 0x7f
g(x) = 0x01
f(x) * g(x) = 0x7f
```

2. AGREGANDO UN MODO DE OPERACIÓN AL CIFRADO DE BLOQUE TIPO “TOY”

EJERCICIO 1

Diseña e implementar un programa para cifrar un mensaje plano m usando tu implementación del cifrado de bloque toy. El mensaje m debe ser una cadena de caracteres ASCII imprimibles.

Tu programa debe pedir al usuario: el nombre del archivo con la llave, el mensaje plano m .

Una vez que tengas el ciphertext c , codifícalo en base64 e imprímelo.

Funciones para codificación Base64

Permite representar un mensaje cifrado (o cualquier dato binario) como texto legible, que se puede copiar, guardar y transmitir sin problemas.

Esta función convierte un arreglo de bytes en una cadena Base64. Funciona de la siguiente forma:

1. Agrupa los datos en bloques de 3 bytes (24 bits).
2. Ese bloque se divide en 4 grupos de 6 bits.
3. Cada grupo de 6 bits se usa como índice en la tabla Base64:
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
4. Si los datos no son múltiplo de 3 bytes, agrega caracteres iguales al final.

```
// Base64 encoding functions
static const char base64_chars[] =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

char *base64_encode(const unsigned char *data, int len, int *out_len) {
    int olen = 4 * ((len + 2) / 3);
    char *out = malloc(olen + 1);
    out[olen] = '\0';

    int i = 0, j = 0;
    while (i < len) {
        unsigned int a = i < len ? data[i++] : 0;
        unsigned int b = i < len ? data[i++] : 0;
        unsigned int c = i < len ? data[i++] : 0;

        unsigned int triple = (a << 16) | (b << 8) | c;

        out[j++] = base64_chars[(triple >> 18) & 0x3F];
        out[j++] = base64_chars[(triple >> 12) & 0x3F];
        out[j++] = (i > len + 1) ? '=' : base64_chars[(triple >> 6) & 0x3F];
        out[j++] = (i > len) ? '=' : base64_chars[triple & 0x3F];
    }

    if (out_len) *out_len = olen;
    return out;
}
```

Sesión de Laboratorio 5: Bit Permutation-Part V

Función de cifrado

Convierte un mensaje de texto normal en su versión cifrada utilizando el Toy Cipher, y luego lo transforma en Base64 para poder enviarlo o almacenarlo de manera segura como texto legible.

```
// Funcion de cifrado por bloque Toy
unsigned char cifrar(unsigned char M, unsigned char k[4], unsigned char S[16], int P[8]) {
    for (int i = 0; i < 3; i++) {
        M ^= k[i];
        M = aplicarSustitucion(M, S);
        M = aplicarPermutacion(M, P);
    }
    M ^= k[3];
    return M;
}

// Funciones para cifrar mensajes
char *cifrarMensaje(const char *msg, unsigned char k[4], unsigned char S[16], int P[8]) {
    int len = strlen(msg);
    unsigned char *cifrado = malloc(len + 1);
    cifrado[0] = (unsigned char)len;

    for (int i = 0; i < len; i++) {
        cifrado[i + 1] = cifrar((unsigned char)msg[i], k, S, P);
    }

    int b64_len;
    char *b64_cifrado = base64_encode(cifrado, len + 1, &b64_len);
    free(cifrado);

    return b64_cifrado;
}
```

Para cifrar se implementó otra función que toma el mensaje completo y envía cada carácter (8 bits) a la función de *cifrar*, la cual cifra bloque por bloque usando en cifrado Toy, esto nos permitió reutilizar las funciones de prácticas pasadas.

El resultado de cada bloque se va agregando al arreglo *cifrado* que contiene al final cada uno de los caracteres cifrados con el cifrado Toy.

Este arreglo se le envía a la función *base64_encode* para que se codifique en base64 y este resultado es el que se le devuelve al usuario.

Las funciones de Permutación y Sustitución son las mismas que se implementaron en sesiones pasadas que solo manejan un bloque de 8 bites.

EJERCICIO 2

Diseña e implementar un programa para descifrar usando tu implementación del cifrado de bloque toy. Esta implementación debe incluir la permutación inversa π^{-1} .

Tu programa debe pedir al usuario: el nombre del archivo con la llave usada para cifrar, el ciphertext c codificado en base64.

Imprime el mensaje recuperado m .

Funciones de decodificación Base64

Restaura los datos originales que habían sido codificados en Base64, permitiendo recuperar correctamente el mensaje cifrado. Esta función toma una cadena Base64 y devuelve los bytes originales.

1. Lee bloques de 4 caracteres Base64 ($4 \times 6 = 24$ bits).
2. Convierte esos 24 bits en 3 bytes originales.
3. Si aparecen caracteres iguales, significa que hay padding y se reduce la cantidad de bytes finales.

```
// Base64 decoding functions
int base64_value(char c) {
    if (c >= 'A' && c <= 'Z') return c - 'A';
    if (c >= 'a' && c <= 'z') return c - 'a' + 26;
    if (c >= '0' && c <= '9') return c - '0' + 52;
    if (c == '+') return 62;
    if (c == '/') return 63;
    return -1;
}

unsigned char *base64_decode(const char *input, int *out_len) {
    int len = strlen(input);
    int padding = 0;

    if (input[len - 1] == '=') padding++;
    if (input[len - 2] == '=') padding++;

    int olen = (len * 3) / 4 - padding;
    unsigned char *out = malloc(olen);

    int j = 0;
    for (int i = 0; i < len; i += 4) {
        int v1 = base64_value(input[i]);
        int v2 = base64_value(input[i+1]);
        int v3 = base64_value(input[i+2]);
        int v4 = base64_value(input[i+3]);

        unsigned int triple = (v1 << 18) | (v2 << 12) |
                               ((v3 & 0x3F) << 6) | (v4 & 0x3F);

        if (j < olen) out[j++] = (triple >> 16) & 0xFF;
        if (j < olen) out[j++] = (triple >> 8) & 0xFF;
        if (j < olen) out[j++] = triple & 0xFF;
    }

    *out_len = olen;
    return out;
}
```

Sesión de Laboratorio 5: Bit Permutation-Part V

Función de Descifrado

Esta función toma como entrada un mensaje cifrado en Base64, lo decodifica y después descifra cada byte con el Toy Cipher para recuperar el mensaje original.

```
// Funcion de descifrado por bloque Toy
unsigned char descifrar(unsigned char C, unsigned char k[4], unsigned char invS[16], int invP[8]) {
    C ^= k[3];
    for (int i = 2; i >= 0; i--) {
        C = aplicarPermutacion(C, invP);
        C = aplicarSustitucion(C, invS);
        C ^= k[i];
    }
    return C;
}

// Funciones para descifrar mensajes
char *descifrarMensaje(const char *cifrado, unsigned char k[4], unsigned char invS[16], int invP[8]) {
    int len;
    unsigned char *cifrado_dec = base64_decode(cifrado, &len);
    int original_len = (int)cifrado_dec[0];
    char *descifrado = malloc(original_len + 1);

    for (int i = 0; i < original_len; i++) {
        descifrado[i] = descifrar((unsigned char)cifrado_dec[i + 1], k, invS, invP);
    }
    descifrado[original_len] = '\0';
    free(cifrado_dec);

    return descifrado;
}
```

Para descifrar se implementó una función que recibe un texto en base64, lo decodifica haciendo uso de la función `base64_decode`.

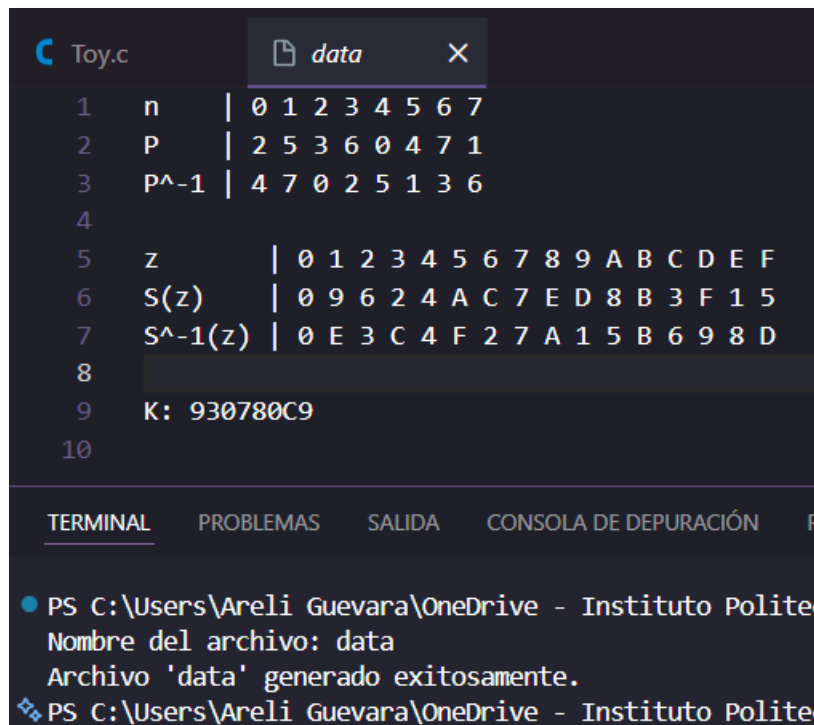
Luego envía carácter por carácter (8 bits) texto decodificado a la función `descifrar` para que este descifre cada bloque y los agregue a un arreglo llamado `descifrado` que será lo que devuelva al usuario como texto descifrado.

PUNTO 3:

Ejecución:

a) Usando tus programas de sesiones anteriores, generamos nuestro archivo con: una llave aleatoria K , una S-box S , la inversa de la S-box S^{-1} , una permutación π , la permutación inversa π^{-1} .

Para esto se creo un programa aparte llamado *genData.c* que retoma todas las funciones de generación de llave, permutación, sustitución y sus inversas para generar un solo archivo como el siguiente.



```

Toy.c
data
1  n | 0 1 2 3 4 5 6 7
2  P | 2 5 3 6 0 4 7 1
3  P^-1 | 4 7 0 2 5 1 3 6
4
5  z | 0 1 2 3 4 5 6 7 8 9 A B C D E F
6  S(z) | 0 9 6 2 4 A C 7 E D 8 B 3 F 1 5
7  S^-1(z) | 0 E 3 C 4 F 2 7 A 1 5 B 6 9 8 D
8
9  K: 930780C9
10

TERMINAL  PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN
● PS C:\Users\Areli Guevara\OneDrive - Instituto Polite
Nombre del archivo: data
Archivo 'data' generado exitosamente.
PS C:\Users\Areli Guevara\OneDrive - Instituto Polite

```

b) Usa tu programa del ejercicio 2.1 para cifrar una cadena de caracteres ASCII imprimibles m .

Tu programa debe usar los datos generados en el punto a).

c) Usa tu programa del ejercicio 2.2 para descifrar el ciphertext c obtenido en b), usando los datos generados en a).

```

1. Cifrar
2. Descifrar
Elige una opcion: 1
Dame el archivo: data
Archivo leído correctamente...
Introduce el mensaje a cifrar:
hola mundo
Mensaje cifrado: CuZhIKPCIV3sYGEA

```

```

1. Cifrar
2. Descifrar
Elige una opcion: 2
Dame el archivo: data
Archivo leído correctamente...
Introduce el mensaje a descifrar:
CuZhIKPCIV3sYGEA
Mensaje descifrado: hola mundo

```


PROBLEMA 4:

Usa el código fuente del ejercicio 2.1 para implementar un programa que cifre un mensaje m usando tu cifrado de bloque toy en modo contador (CTR).

Recuerda que el contador IV debe generarse aleatoriamente cada vez que cifres un mensaje m .

```
// Funciones para cifrar mensajes
char *cifrarMensaje(const char *msg, unsigned char k[4], unsigned char S[16], int P[8]) {
    int len = strlen(msg);
    unsigned char *cifrado = malloc(len + 1);
    unsigned char IV = (unsigned char)(rand() % 256);

    for (int i = 0; i < len; i++) {
        cifrado[i] = (cifrar(IV+i, k, S, P)) ^ (unsigned char)msg[i];
    }

    int b64_len;
    char *b64_cifrado = base64_encode(cifrado, len + 1, &b64_len);
    free(cifrado);

    return b64_cifrado;
}
```

La función `cifrarMensaje()` implementa un cifrado basado en el modo Counter Mode (CTR), adaptado al cifrado Toy. En lugar de cifrar directamente cada byte del mensaje, la función genera primero un IV (vector de inicialización) aleatorio de 8 bits.

Este IV se utiliza para construir una secuencia de valores pseudoaleatorios al enviarlos al cifrado de bloque Toy.

Cada uno de estos valores cifrados se combina con el byte correspondiente del mensaje mediante la operación XOR y se le asigna como el valor i del mensaje cifrado.

Finalmente, el resultado binario del cifrado se convierte en una cadena imprimible mediante Base64, lo que facilita su almacenamiento o transmisión.

Ejecución:

```
Dame el archivo: data
Archivo leído correctamente...
Introduce el mensaje a cifrar:
Areli Alejandra
Mensaje cifrado: 4t5QDPqq5Irc7ItORZ4AbAAA
```